MT72 - Bruit de Perlin

Clément BARATIN

17 avril 2026

1 Introduction

L'objectif de ce TP était de choisir un sujet mettant en relation mathématiques et informatique de manière concrète afin de mobiliser des connaissances concernant les deux sujets. Nous avons choisi de présenter un algorithme de génération procédurale basé sur une valeur aléatoire : **Le bruit de Perlin**.

Développé par Ken Perlin en 1985, le bruit de Perlin utilise les propriétés des gradients pour générer un rendu pseudo aléatoire permettant alors de générer un environnement concret. Nous détaillerons dans le compte-rendu qui suit l'implémentation de celui-ci, le code associé et l'utilisation du programme fourni.

Nous traiterons dans ce compte rendu uniquement d'une application dudit algorithme dans un espace à 2 dimensions. Notons qu'une extrapolation dans un espace à n plan(s) est tout à fait envisageable.

2 Algorithme

2.1 Définition de l'espace

Considérons le plan réel $P = \mathbb{R}^2$. Nous subdivisons ce plan par une grille régulière dont les noeuds correspondent aux points à coordonnées entières de $\mathbb{R}^2$.

À tout point $p = (x, y) \in P$, nous associons les quatre sommets de la cellule de la grille (le carré unité) dans laquelle il est inscrit. Ces sommets sont définis par les parties entières inférieures et supérieures des coordonnées de p.

Par exemple, pour le point $\lambda = (3.2, 4.5)$, l'ensemble des sommets associés est :

$$S = \{(\lfloor x \rfloor, \lfloor y \rfloor), (\lceil x \rceil, \lfloor y \rfloor), (\lfloor x \rfloor, \lceil y \rceil), (\lceil x \rceil, \lceil y \rceil)\}$$

Soit, dans ce cas précis : $S = \{(3, 4), (4, 4), (3, 5), (4, 5)\}$.

2.2 Table de permutation

La table de permutation représente la variable aléatoire du bruit. La table est de taille 512 de sorte que les éléments d'indice $0 \leq i \leq 256$ soient les valeurs 0..256 distribuées de manière aléatoire, de même pour les indices $256 \leq i \leq 512$.

2.3 Vecteurs de gradient et produit scalaire

À chaque nœud (i, j) de la grille, on associe un vecteur de gradient pseudo-aléatoire $\vec{g}_{i,j}$. Pour garantir la cohérence du bruit, ce vecteur est extrait d'un ensemble prédéfini en utilisant la table de permutation pour "hasher" les coordonnées (i, j).

Pour un point $p = (x, y)$, on calcule ensuite quatre vecteurs de distance $\vec{d_s}$ reliant chaque sommet $s \in S$ au point p :

$$\vec{d_s} = p - s$$

La contribution de chaque sommet est déterminée par le produit scalaire entre le gradient au sommet et le vecteur distance associé :

$$v_s = \vec{g}_s \cdot \vec{d_s}$$

2.4 Interpolation et fonction de lissage

Le calcul final du bruit au point p consiste à interpoler les quatre valeurs v_s. Pour obtenir un rendu visuel continu et éviter les cassures de pente aux limites des cellules, on utilise une fonction de lissage (fade function) :

$$f(t) = 6t^5 - 15t^4 + 10t^3$$

[1]

Soient $u = f(x - \lfloor x \rfloor)$ et $v = f(y - \lfloor y \rfloor)$ les coefficients d'interpolation :

— $v_{top} = lerp(v_{(\lfloor x \rfloor, \lceil y \rceil)}, v_{(\lceil x \rceil, \lceil y \rceil)}, u)$
— $v_{bottom} = lerp(v_{(\lfloor x \rfloor, \lfloor y \rfloor)}, v_{(\lceil x \rceil, \lfloor y \rfloor)}, u)$
— $B(p) = lerp(v_{bottom}, v_{top}, v)$

où $lerp(a, b, t) = a + t(b - a)$. L'interpolation linéaire est une simplification d'un bruit de Perlin de nouveau appliqué sur une valeur définie t. L'intérêt est ici de ne pas avoir de gap dans le rendu.

3 Génération de carte par octaves

La classe `PerlinMapGenerator` constitue la couche applicative supérieure de l'implémentation. Son rôle est de générer une carte bidimensionnelle de valeurs de bruit en combinant plusieurs appels au générateur de bruit de base selon une technique dite par **octaves**.

3.1 Paramètres de la classe

La classe expose les paramètres suivants :

— `width`, `height` : dimensions en pixels de la carte générée (par défaut 5000×5000).
— `octave` : nombre d'octaves à superposer (par défaut 6).
— `scale` : facteur d'échelle global appliqué aux coordonnées d'échantillonnage.
— `base_frequency` : fréquence de la première octave.
— `base_amplitude` : amplitude de la première octave.
— `persistence` : facteur multiplicatif de l'amplitude entre chaque octave consécutive.
— `lacunarity` : facteur multiplicatif de la fréquence entre chaque octave consécutive.

La carte est stockée sous la forme d'un tableau bidimensionnel `perlin_values` de flottants, initialisé à 0.0 pour chaque cellule.

3.2 Principe des octaves

L'idée centrale est de superposer N couches de bruit de Perlin, chacune caractérisée par une fréquence et une amplitude propres. Pour la i-ème octave ($0 \leq i < N$), on définit :

$$f_i = f_0 \cdot \lambda^i \qquad a_i = a_0 \cdot \rho^i$$

où f_0 est la fréquence de base, a_0 l'amplitude de base, λ la lacunarité et ρ la persistance. En pratique, $\lambda > 1$ (typiquement 2) fait croître la fréquence d'octave en octave, introduisant des détails de plus en plus fins, tandis que $\rho < 1$ (typiquement 0.5) atténue progressivement leur contribution.

La valeur accumulée au point (x, y) est alors :

$$V(x, y) = \sum_{i=0}^{N-1} a_i \cdot B\Big(\frac{x}{W} \cdot s \cdot f_i,\ \frac{y}{H} \cdot s \cdot f_i\Big)$$

où B désigne la valeur de bruit de Perlin, W et H les dimensions de la carte, et s le facteur d'échelle[2]. Notons ici que nous aurions pu nous servir du code du TP pour traiter les données de la carte sous forme matricielle. Or la classe du cours étant trop complète pour notre utilisation nous avons préféré utiliser les vecteurs de la librairie standards.

1. Cette fonction est tout à fait modifiable dans le programme, notons que la version utilisée est celle conseillé par Ken Perlin.

2. La somme telle qu'elle est largement optimisable d'un point de vue logiciel, ceux-ci sera l'objet d'une mise à jour du code.

3.3 Normalisation

Une fois toutes les octaves accumulées, les valeurs obtenues peuvent dépasser l'intervalle $[-1, 1]$. On calcule l'amplitude maximale théorique :

$$A_{max} = \sum_{i=0}^{N-1} a_0 \cdot \rho^i$$

Chaque valeur de la carte est ensuite divisée par A_{max}, ce qui garantit que le résultat final est borné dans $[-1, 1]$, indépendamment du nombre d'octaves choisi. Cette étape est indispensable pour obtenir une carte normalisée exploitable directement, par exemple pour un rendu en couleur ou un calcul d'altitude (les deux sont souvent liés).

3.4 Fonction de lissage paramétrable

La méthode `generate` reçoit en paramètre une référence vers une instance de `ISmoothingFunction`, qui est transmise à chaque appel du générateur de bruit de base. Ce mécanisme d'injection de dépendance permet de choisir à la volée la fonction de lissage utilisée lors de l'interpolation (voir section 2.4), sans modifier la logique de génération par octaves.

4 Utilisation du programme

4.1 Lancement du programme

Pour lancer le programme `Perlinpinpin` nous conseillons l'utilisation du gestionnaire de compilation `CMake` qui permet de générer automatiquement tout les fichiers nécessaires à l'exécution du programme. Pour ce faire, à la racine du projet, générez une build avec la commande :

```
cmake -S . -B build
```

Transformez la build en exécutable machine :

```
cmake --build build
```

Et exécutez la commande :

```
./build/perlinpinpin
```

Vous devriez voir une image représentant un monde factice être générée à la racine du projet.

4.2 Modification des paramètres

Comme décrit la partie dédié à la génération de carte (3.1) il est possible de modifier les données d'entrée de notre programme afin d'obtenir des résultats différents, notons que la modification, de la taille de l'image dans la classe `PerlinMapGenerator` doit aussi être effecutée dans l'appel de la fonction générant l'image dans la fonction `main` lors de l'appel de `generate_png_image`.

Il est aussi possible de modifier le rendu des couleurs en modifiant les niveaux définie dans le fichier `Biome.h`.

4.3 Documentation

La documentation technique du code se trouve à la racine du projet et le code source est aussi disponible à l'adresse : https://github.com/waterfox-dev/perlinpinpin